

# Chapter 12 Summary — Sequence Models and LLM Agents in Strategic Settings

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

## Contents

|                                                                             |          |
|-----------------------------------------------------------------------------|----------|
| <b>12 Chapter 12 — Sequence Models and LLM Agents in Strategic Settings</b> | <b>1</b> |
| 12.1 Why reframe reinforcement learning as sequence prediction . . . . .    | 1        |
| 12.2 The luck-versus-skill trap . . . . .                                   | 2        |
| 12.3 ARDT — conditioning on what you can guarantee . . . . .                | 2        |
| 12.4 What return conditioning actually does in poker . . . . .              | 4        |
| 12.5 Language models as strategic agents . . . . .                          | 5        |
| 12.6 Exploitation, adaptation, and the limits of the toy game . . . . .     | 6        |
| 12.7 Key takeaways for the thesis synthesis . . . . .                       | 7        |

## 12 Chapter 12 — Sequence Models and LLM Agents in Strategic Settings

### 12.1 Why reframe reinforcement learning as sequence prediction

Classical RL optimises: estimate a value function, back up rewards, improve the policy. The Decision Transformer (Chen et al., 2021) proposes something different — treat the whole problem as **supervised sequence modelling**. Feed a GPT-style model a trajectory of triples

$$(\hat{R}_1, s_1, a_1, \hat{R}_2, s_2, a_2, \dots)$$

where  $\hat{R}_t = \sum_{t' \geq t} r_{t'}$  is the *return-to-go*, and train it to predict  $a_t$  from everything up to  $s_t$ . There is no Bellman backup and no policy-improvement step. At inference you simply **condition**: ask for a high return-to-go and the model produces the actions that historically preceded such returns.

The appeal for this thesis is that it works entirely **offline**, on a fixed dataset — the regime Chapter 13’s Playtech logs live in, where self-play is unavailable.<sup>1</sup>

---

<sup>1</sup>Chen et al., *Decision Transformer: Reinforcement Learning via Sequence Modeling*, NeurIPS 2021 (arXiv:2106.01345).

## 12.2 The luck-versus-skill trap

The reframing hides an assumption: that the return-to-go is something the agent *earned*. In a stochastic environment it is not. Paster et al. (2022) show that return-conditioned policies systematically chase luck, and the smallest possible example makes it obvious.

Consider a one-step bandit. Action A pays 0.5 deterministically. Action B pays 1.0 with probability 0.4 and 0 otherwise, so  $\mathbb{E}[B] = 0.4 < 0.5$ . Action A is EV-optimal. But the *only* way a trajectory ever achieves a return of 1.0 is to take B **and get lucky**. Condition on “return = 1.0” and you recover exclusively B-trajectories:

$$P(a = B \mid \hat{R} = 1.0) = 1.00 \quad (\text{measured})$$

The return-conditioned learner confidently selects the action with the *worse* expected value. This is not a bug in the model; it is what conditioning on an outcome means when the outcome is partly noise.<sup>2</sup>

## 12.3 ARDT — conditioning on what you can guarantee

The fix is to condition not on the return that *happened* but on the return the protagonist can **guarantee against a worst-case opponent** — the minimax return-to-go. ARDT (Tang et al., 2024) estimates it with **expectile regression**, which minimises the asymmetric squared loss

$$L_{\text{ER}}^{\alpha}(u) = \mathbb{E}_u[|\alpha - \mathbf{1}(u > 0)| \cdot u^2]$$

whose limits give the operators you need:  $\alpha \rightarrow 0$  recovers the **minimum** and  $\alpha \rightarrow 1$  the **maximum**. The trajectories are relabelled with the estimated minimax return and a standard DT is trained on the relabelled data.

Two details of the published method matter, and both were confirmed by reading the source rather than the summary. First, the pessimistic side is **low**  $\alpha$  (the raw step’s claim that  $\tau = 0.9$  is pessimistic is inverted); the paper itself runs  $\alpha = 0.01$ . Second — the important one — the relabel target is

$$\tilde{R}_t = \tilde{Q}_{\nu}(s_t, a_t)$$

a state-**action** value produced by two *coupled* estimators alternately fitted with  $\alpha$  and  $1 - \alpha$  losses. A state-only value  $V(s)$  cannot distinguish “this state is bad” from “*this action* in this state is bad”, which is exactly the discrimination the method relies on.

Measured on Kuhn with a deliberately simplified state-only proxy, the relabel target moves monotonically with  $\tau$  as theory requires — but exploitability is *lowest* on the optimistic side, which is the signature of

---

<sup>2</sup>Paster, McIlraith & Ba, *You Can’t Count on Luck: Why Decision Transformers and RvS Fail in Stochastic Environments*, NeurIPS 2022 (arXiv:2205.15967).

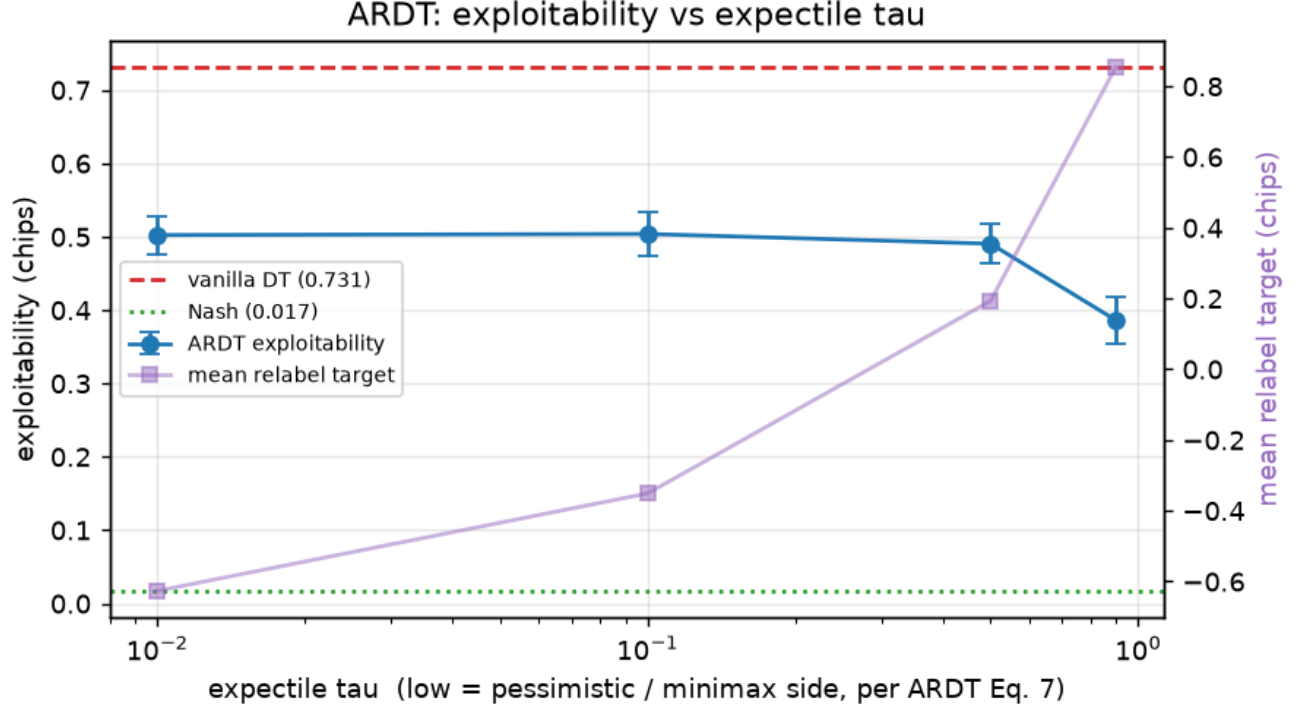


Figure 1: ARDT exploitability against the expectile tau, with the mean relabel target on the second axis.

the missing action argument rather than evidence against ARDT.<sup>3</sup>

<sup>3</sup>Tang, Zhang, Gu et al., *Adversarially Robust Decision Transformer*, NeurIPS 2024 (arXiv:2407.18414) — Section 3 and Algorithm 1.

## 12.4 What return conditioning actually does in poker

Kuhn Poker’s equilibrium and exploitability are exactly computable, so the question can be settled rather than argued. The result is that conditioning **changes** the policy substantially but does not **steer** it: exploitability is flat across the range of real target returns, with a sharp collapse at one specific value — the modal payoff, which in Kuhn is the payoff of folding.

The natural explanation is that in a four-payoff game the *magnitude* of the return encodes which betting line was played, while its *sign* encodes who held the better card. Conditioning therefore selects the shape of the hand rather than the quality of the play.

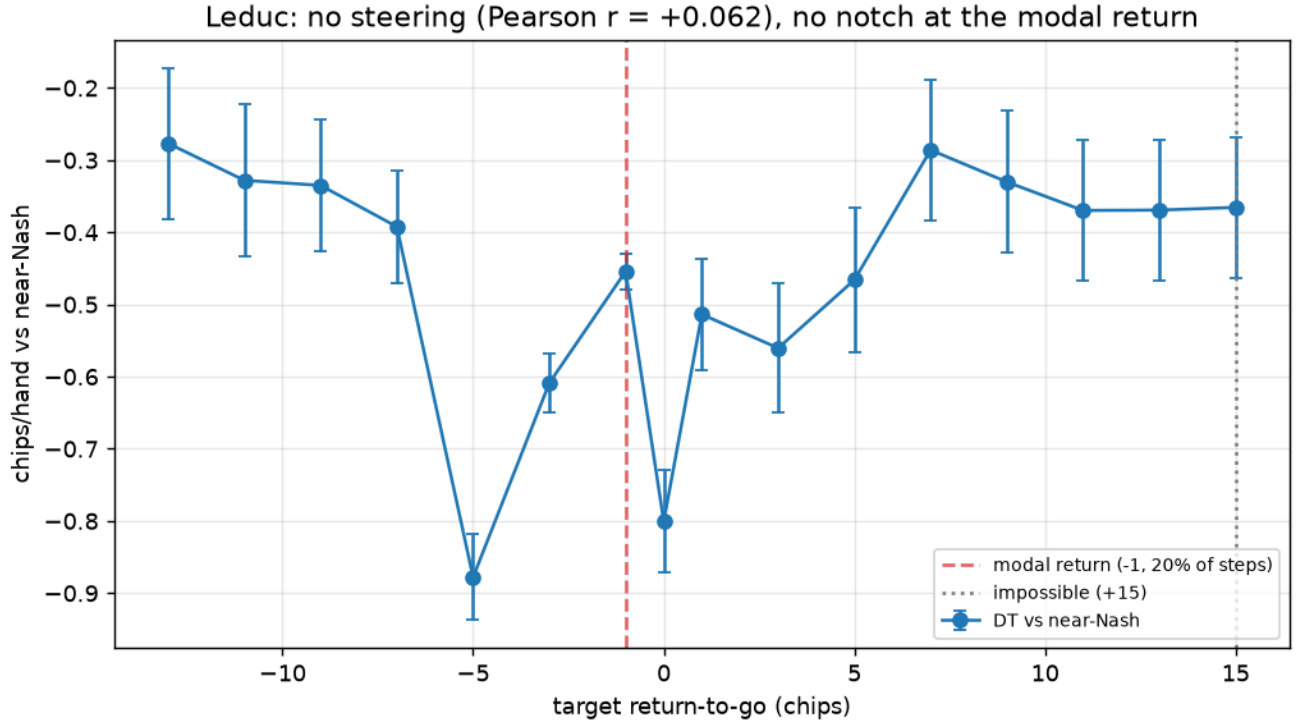


Figure 2: Decision Transformer performance against the conditioned target return on Leduc Hold’em, with standard errors.

Leduc Hold’em tests that explanation with fifteen payoff values instead of four, two betting streets and a board card. **The collapse disappears exactly as the explanation predicts — and steering still does not appear** (Pearson  $r = +0.062$ ). So the payoff-alphabet account explains the collapse but not the failure. The failure is more fundamental: in a zero-sum imperfect-information game the realised return is dominated by the opponent’s private card and actions, which the protagonist does not control. No amount of payoff granularity makes an uncontrollable signal steerable.

This is the argument that carries into Chapter 13. On fixed logs, a return-conditioned Decision Transformer is the wrong instrument — and the value of ARDT’s relabeling is precisely that it replaces an uncontrollable conditioning target with a controllable one.

## 12.5 Language models as strategic agents

The second paradigm skips training altogether: describe the rules in English and let a language model play. Measured against the same exact ruler, LLMs are **honestly exploitable** — but the interesting result is *where* they fail.

They get hand **ranking** right and mixing **frequencies** wrong. Every backend tested value-bets the King with probability 1.00 where equilibrium mixes at 0.68, and none bluffs the Jack at all under a plain prompt. A 7B model matches a 20B model, because the game rewards correct mixing rather than knowledge or scale.

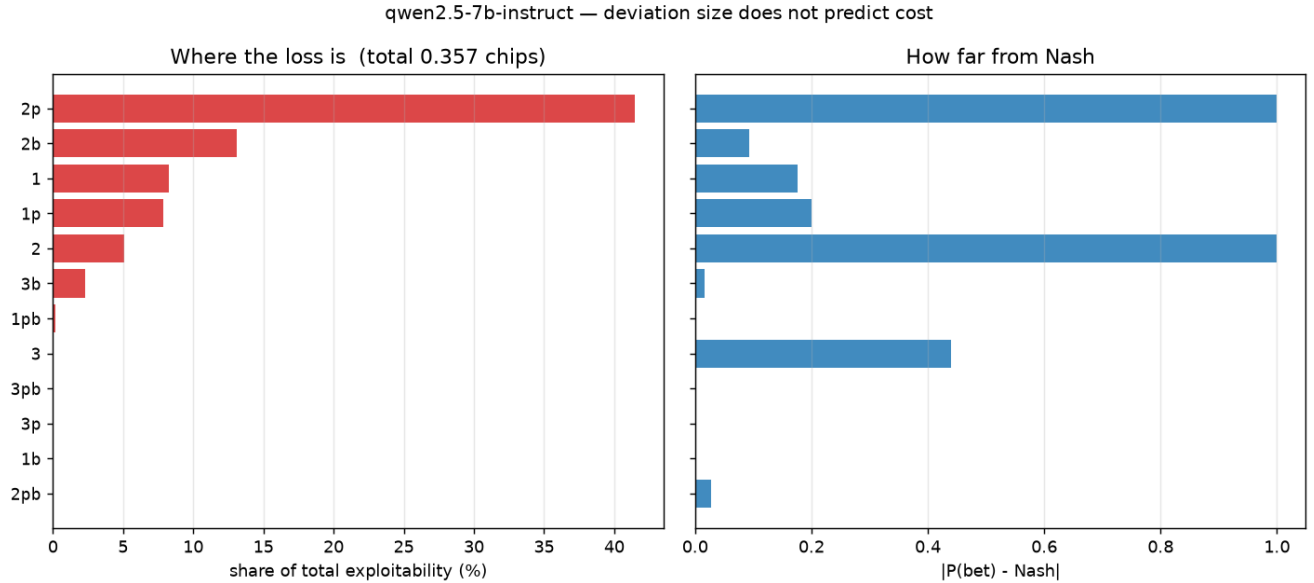


Figure 3: Each information set’s share of total exploitability, beside its deviation from equilibrium.

Decomposing the loss per decision overturns the obvious reading. The King “failure” — the largest visible deviation from equilibrium — costs **0.1%** of the total loss, because over-betting a hand that is never behind is nearly free. A single Queen decision costs **41.4%**. Deviation magnitude and cost are almost uncorrelated, which is the concrete argument for reporting per-decision diagnostics rather than a single exploitability number in the Chapter 14 evaluation framework.

Asking a model what it intends is *worse* than measuring what it does. Scored as strategies, the frequencies these models **state** are 2.6× and 4.0× more exploitable than the strategies they actually **play**. One model answers “50%” almost everywhere; the other answers “near 0%” almost everywhere, including with the strongest hand. Behavioural probing is not merely more convenient than introspection here — it is more accurate.<sup>4</sup>

<sup>4</sup>Guertler et al., *TextArena: A Framework for Text-Based Game Environments*, 2025 (arXiv:2504.11442).

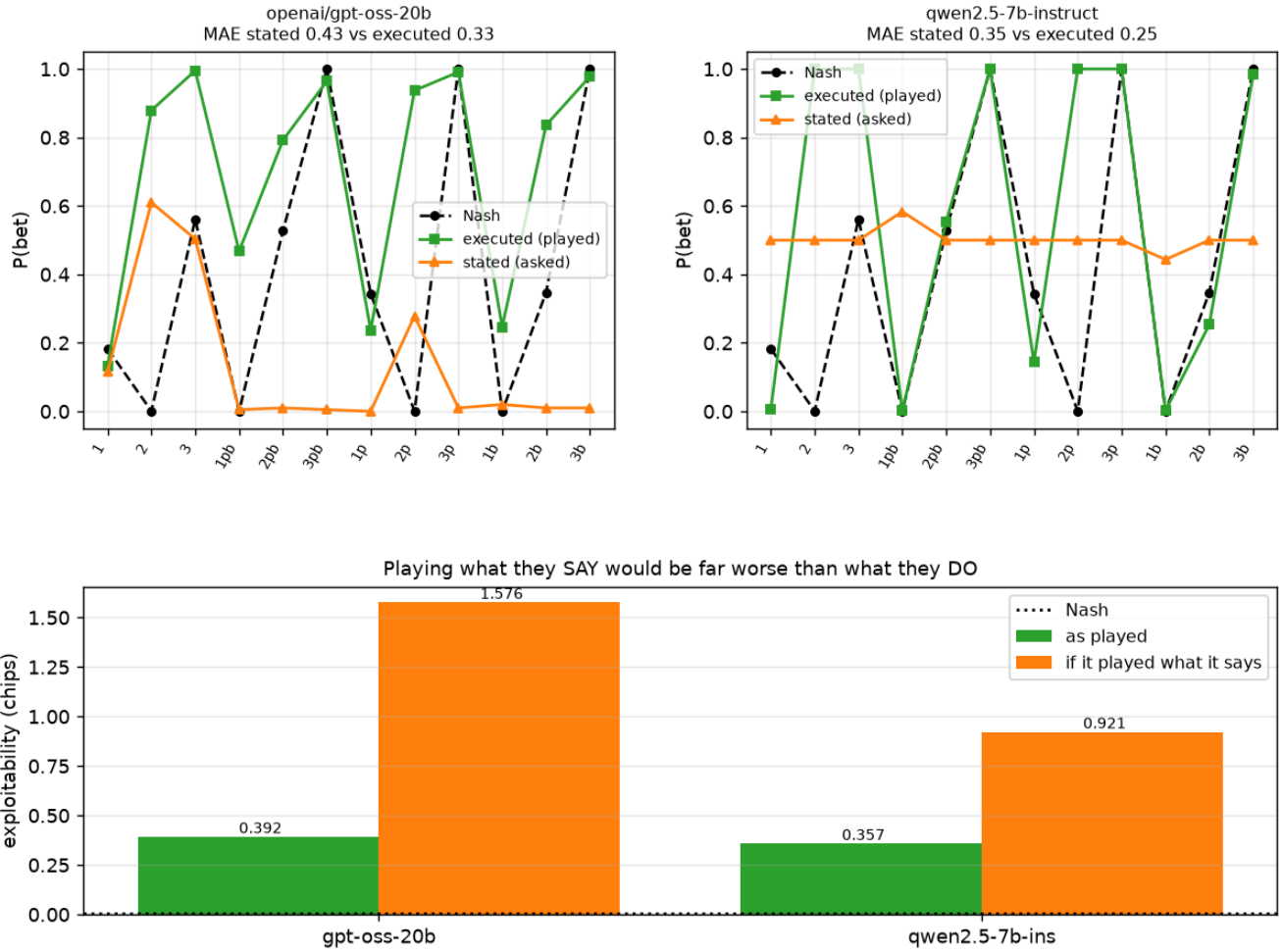


Figure 4: Stated versus executed betting frequencies for two models, with the exploitability of each strategy.

## 12.6 Exploitation, adaptation, and the limits of the toy game

Exploitability measures how a perfect adversary punishes you; it says nothing about how well you punish weakness. Measured against a zoo of deliberately exploitable archetypes, the language model **exploits 61% harder than equilibrium play while being 59× more exploitable** — but the entire gain comes from passive and random opponents. Against the two most competent archetypes it does *worse* than equilibrium. That is the safe-exploitation trade-off of the thesis’s second contribution, observed rather than assumed.

Given the observed history of a session in context, the model does **not** adapt: it captures 83% of the available exploitation against a trivially passive opponent from the first half of the session onward, and never improves (mean learning  $-0.22$ ). What looks like opponent modelling is a fixed loose-aggressive prior.

Finally, the toy game flatters these models. One street and one board card later, the LLM is statistically indistinguishable from the Decision Transformer, cannot beat weak opponents at all, and wants to take

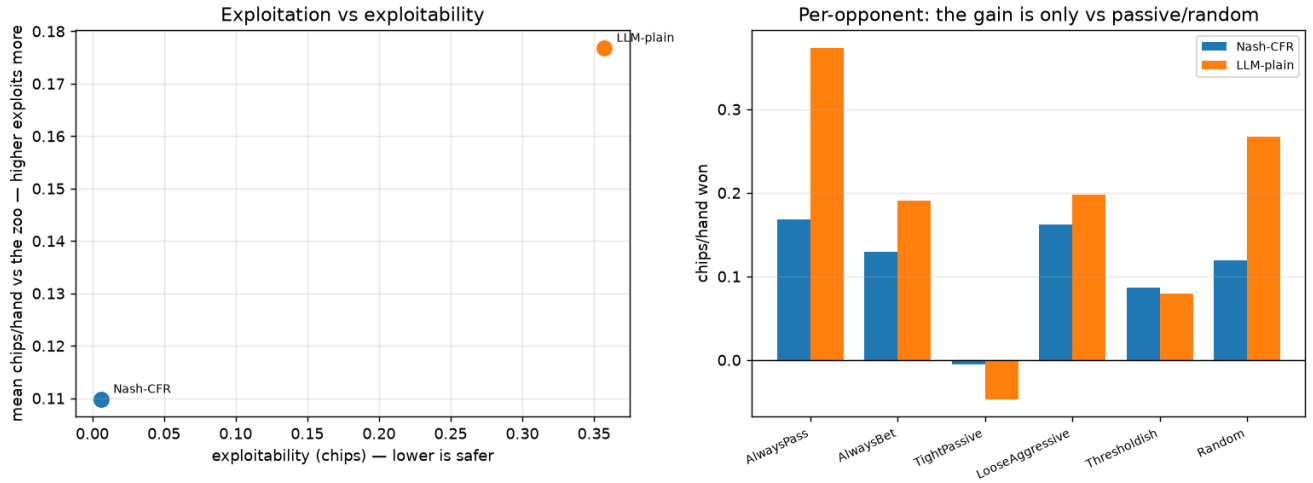


Figure 5: Exploitation against exploitability, with the per-opponent breakdown.

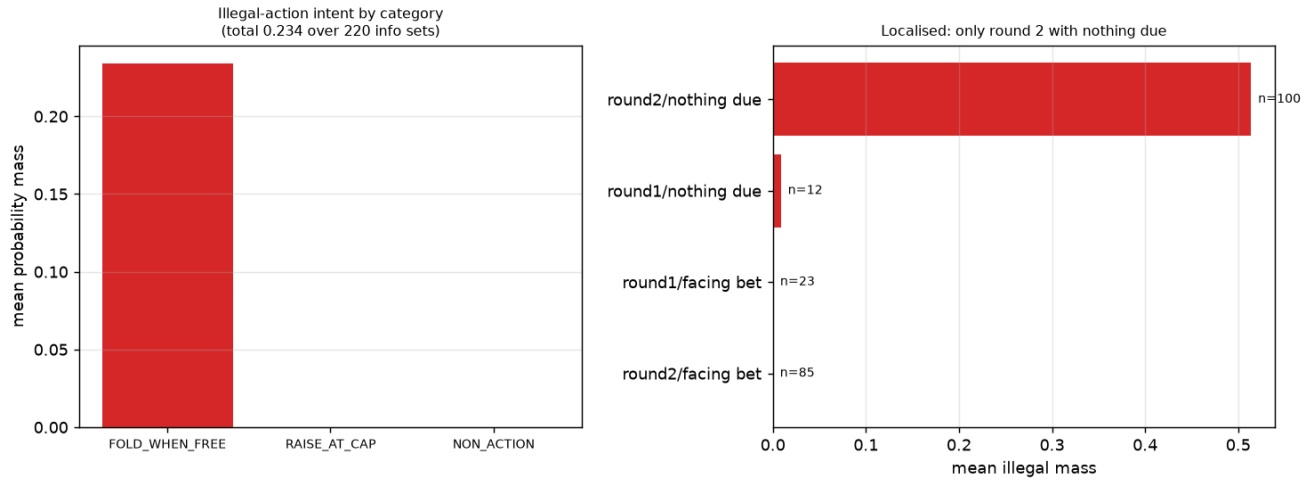


Figure 6: Illegal-action intent by category and by situation on Leduc Hold'em.

an illegal action with roughly a quarter of its probability mass. That failure is a *single* misconception rather than diffuse confusion — it wants to **fold when checking is free**, almost exclusively on weak unpaired hands against a high board, while never once attempting to raise past the betting cap. Any optimism drawn from the toy game should be read as a property of the toy game.

## 12.7 Key takeaways for the thesis synthesis

- **Conditioning on a quantity the agent does not control cannot steer it.** Demonstrated on two games; it is why ARDT's relabeling — not the Decision Transformer itself — is what transfers to fixed poker logs.
- **The published ARDT relabels with a state-action value.** Reproducing the method's benefit requires  $\tilde{Q}(s, a)$ , not  $V(s)$  — the identified, evidence-backed fix for Chapter 13.
- **A scalar score ranks methods but does not diagnose them.** One decision carried 41.4% of

the loss while the most conspicuous deviation carried 0.1%.

- **Measure behaviour, not self-report.** Verbalised strategy was substantially worse than played strategy for every model tested.
- **In-context opponent adaptation did not emerge**, so an explicit opponent model is required rather than assumed — directly relevant to the first thesis contribution.
- **Measurement protocol is part of the result.** At greedy decoding a language model plays a pure strategy and every measured frequency degenerates to 0 or 1; four separate findings in this chapter turned out to be measurement artefacts, each caught by a cheap check against an exactly computable quantity.